How OTP Autofill protects your data

Security, privacy and algorithms explained in plain English

LOCAL CHROME EXTENSION | CURRENT IMPLEMENTATION

It reduces exposure by processing email locally, keeping candidate codes temporary, and filling only on sites you approve. You still need to trust the extension, Chrome, your device and the receiving website.

The key trade-off: Gmail read-only permission can read your mailbox broadly. The current code narrows its own searches to recent verification messages; Google does not limit this grant to OTP messages.

What protects you

Google sign-in, direct Gmail API calls, per-site grants, conservative matching and no application OTP history.

What remains exposed

A filled OTP is visible to the receiving site. A compromised extension or device could misuse sensitive data.

2 min
Attempt deadline

3 min
Maximum eligible email age

0
Developer backend in this build

Read this as an explanation of implemented controls, not a security certification. No penetration test or statistically measured accuracy is claimed. All sample domains and OTPs in this guide are fictional.

Basis: current source and validated local build. Platform references and source map: page 11.

Your OTP makes one short trip

Google sends the email data to your extension; matching happens on your device.

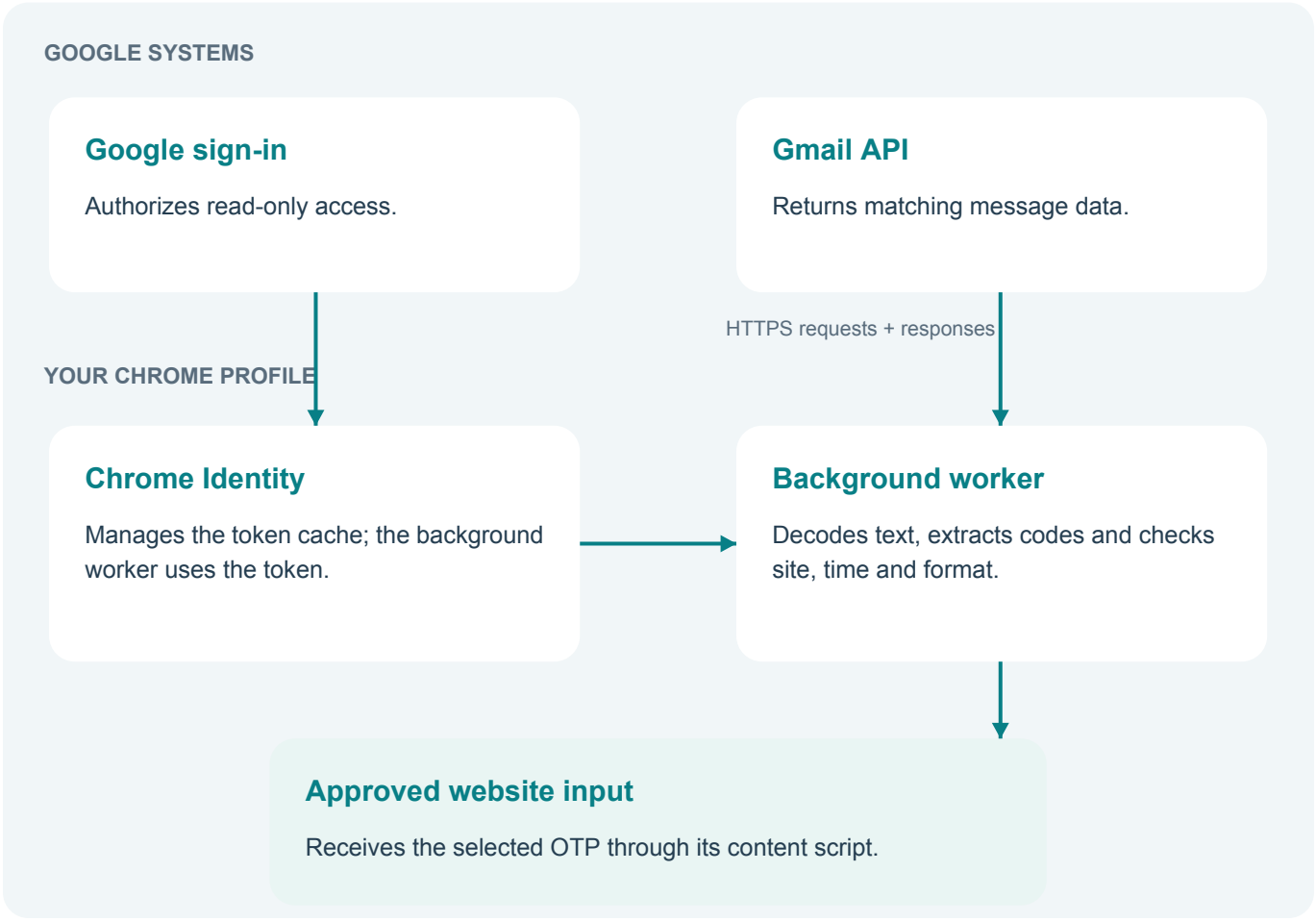


Figure 1. Implemented data path. Only the selected code goes into the form; the token and email body stay out of the page.

Who can see what?

- **Google:** sign-in, permission decisions and the Gmail API requests, including the search query.
- **The extension:** returned message data and candidate OTPs during processing. No developer server, analytics service or AI model is called by this build.
- **The approved website:** the filled OTP and normal form activity. Site scripts can read the field and its input/change events.

Important boundary: isolated extension JavaScript protects its variables, but the website and extension share the page DOM. An OTP written into the DOM is no longer hidden from that site.

Code: src/background/gmail.ts; src/background/coordinator.ts; src/content/autofill.ts. Platform: [1], [4].

Google grants access, not your password

OAuth gives the extension a token: a temporary credential used with Google APIs.

1

You choose Connect

Google presents account and consent screens.

2

Chrome Identity gets a token

The worker checks that Gmail read-only scope was granted.

3

Connection is checked

A Gmail profile request must succeed before connected is saved.

Access or identifier	Meaning in this build
gmail.readonly	Broad read access to email messages and settings. This scoped token does not authorize sending, modifying or deleting mail.
Website permission	Enables scripts for the approved host. The grant persists until removed; it covers that host's pages, not just one OTP field. HTTPS is used, with loopback HTTP reserved for local testing.
OAuth client ID	Public app configuration tied to this extension ID. It is not a password or a token. No client secret is embedded.
Extension ID / public key	Identifies the extension. A public manifest key can stabilize its ID; it does not encrypt OTPs. This local installation currently uses its folder-derived ID.

A token is sensitive. Application code does not save or log it, and does not send it to page JavaScript. Chrome manages its cache. Malware, a stolen live token or a malicious extension update can still create risk.

Current local setup: External audience, Testing status, owner as the test user. The owner reported Gmail connected. Testing mode and a successful connection are not independent security approval.

API requests use bearer authorization over HTTPS, omit cookies, refuse redirects and request no-store caching. These controls still depend on a trustworthy browser and device.

Code: src/background.ts; src/background/gmail.ts; manifest.config.mjs. Platform: [1], [2], [5].

What is kept, and for how long?

Reducing retained data limits what can be exposed later. Some metadata still remains.

Location	Data present	Lifetime / control
Worker memory	Candidate OTPs, message IDs, sender and subject for matching; response bodies during processing.	Candidates cleared on fill, cancellation, timeout or disconnect. Attempts expire at 2 minutes. Bodies are not persisted.
Local settings	Autofill, connected and revocation-pending flags.	Persists until changed or extension data is removed. No OTP or token stored here.
Session status	Flow ID, hostname, timestamps and state; ambiguous-candidate sender domain, code length and time.	Cleared on worker start and tab removal. Chrome also clears session storage on reload/restart. Metadata can outlive the OTP attempt.
Chrome Identity	Google access-token cache.	Chrome manages expiration; disconnect requests revocation and clears cached authorization.
Chrome site grants	Approved host permissions.	Persists until you remove permission or uninstall.
Website / Google	The website sees the filled OTP; Google retains mail under its own account behavior.	Extension cleanup does not delete the input value, website copies or the source email.

The application keeps no OTP history

- Both extension storage areas use **TRUSTED_CONTEXTS**, restricting content-script access to stored settings and status.
- There is no application use of Chrome sync storage. Hostnames and sender-domain metadata still deserve privacy protection.
- Clearing JavaScript references is best-effort cleanup, not certified memory erasure. Browser/OS memory, swap or crash artifacts are outside this guarantee.

Token storage is delegated to Chrome; this build does not implement its own encrypted OTP vault or claim end-to-end encryption. API transport uses HTTPS.

Code: src/background.ts; src/background/settings.ts; src/background/coordinator.ts. Platform: [2], [3].

First, find a real verification task

The algorithm is deterministic: explicit rules and regular expressions, with no AI model.

A. Detect the target

Inspect the first 500 inputs. Require one empty, usable target. Use one-time-code autocomplete, labels, names or placeholders. Reject phone, card, CVV, payment, invoice, postal and promo-like fields.

B. Start a bounded Gmail search

Search newer than detection minus 15 seconds for verification phrases; exclude spam/trash. Up to 10 message IDs. Extra result pages cause a conservative wait/retry.

C. Decode email data locally

Fetch up to 3 bodies at once. Validate structures and decode base64url/MIME. Collect text; convert HTML without rendering it. Skip named attachments during extraction.

D. Extract plausible code candidates

Require a recognized OTP phrase and token within 65 characters. Reject invoice/reference/phone contexts and common words. Preserve case; remove grouped-digit spaces. Keep up to 8 candidates per email.

Figure 2. The discovery pipeline. Each stage can safely stop without filling. All filtering happens before a code is delivered.

Supported shape	Examples / boundaries
Numeric	4-8 digits; grouped examples: 482 193 or 4821 9307.
Letters / mixed tokens	4-12 characters under extraction rules; uppercase or mixed letter-digit tokens. Some hyphenated tokens are supported within the same length budget.
Split or dynamic fields	4-8 one-character boxes with OTP context; newly added DOM fields are detected. Unsupported patterns, shadow DOM and iframe-only forms can require manual entry.

Code: src/content/detector.ts; src/core/email-parser.ts; src/background/gmail.ts. Examples are fictional.

One code must pass every gate

The matcher prefers waiting or asking you over making a risky guess.

SITE Sender and page share the same registrable domain, using public/private suffix rules.

TIME Email is recent, near this attempt, and not too far in the future.

FORMAT Length, numeric requirement and supported field pattern all agree.

CHOICE Deduplicate identical codes; require one eligible code with a high rule score.

Rule scores are not probabilities



Figure 3. Fixed extraction scores from the code. 0.98 does not mean 98% correct, and 0.95 is not a measured accuracy target.

Eligible result	Action
No qualifying code	Wait within the attempt deadline.
One unique code, score at least 0.95	Allow automatic delivery after the final browser checks. Current extraction rules produce 0.98 for this path.
Multiple distinct codes, or score 0.85	Show candidate metadata for explicit selection, or let the user type the code. All domain, time and format checks still apply.

Domain example: login.example.com and verify@accounts.example.com share example.com. example.com.evil.test does not. However, a compromised sibling host under example.com can still share that parent. The From header is not proof of sender authentication; this matcher does not validate DKIM/SPF/DMARC results.

Code: src/core/matcher.ts; src/core/email-parser.ts; src/core/validation.ts. Rules are implementation policy.

Short windows reduce stale-code risk

These values are code settings, not measured delivery speed or the issuer's OTP expiry.

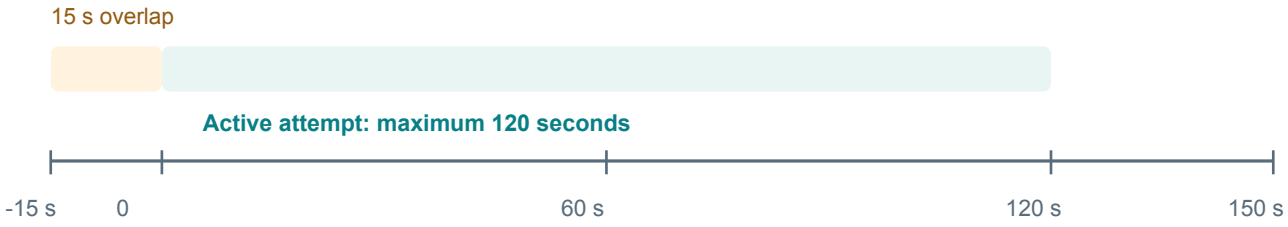


Figure 4. Detection starts at 0. Messages may arrive up to 15 seconds earlier; every candidate must also be at most 180 seconds old and at most 15 seconds in the future.

Polling slows down instead of running forever

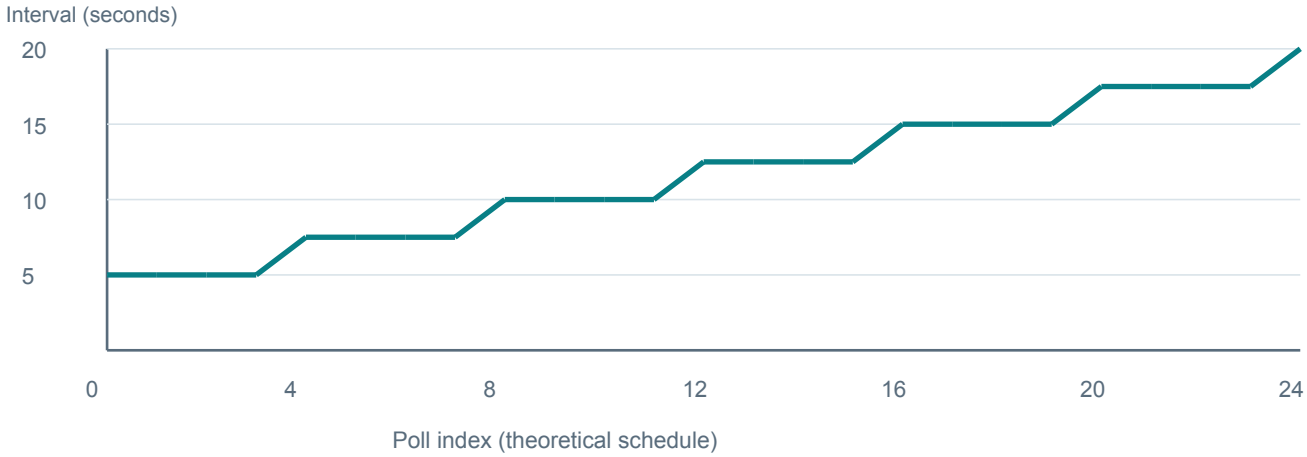


Figure 5. Normal delay = min(5 + 2.5 x floor(index / 4), 20) seconds. The two-minute deadline can end a real attempt before later indices are reached. This is not a latency benchmark.

Control	Implementation
Request / retry limits	10-second HTTP timeout; one refresh retry after HTTP 401. Temporary network/server errors and rate limits back off within the attempt deadline.
Request coordination	At least 5 seconds between search starts, enforced per attempt and by a worker-wide gate. Message bodies may be fetched in parallel batches of 3.
Resource bounds	Up to 40 eligible candidates retained per attempt. Extraction examines up to 100,000 text characters; MIME traversal is depth-limited. Response size is checked after reading, not streamed as a hard download cap.

Code: src/core/lifecycle.ts; src/core/types.ts; src/background/gmail.ts; src/background/coordinator.ts.

Recheck the page before filling

Checks are repeated after Gmail responds, so a late result cannot blindly fill an old form.

Boundary	Required check
Content script to worker	Message comes from this extension, top frame and a document ID. Context shape and constraints are validated.
Worker to browser tab	Host permission exists; tab is active; sender URL equals current tab URL; context hostname agrees. Settings must allow autofill and Gmail must be connected.
Before delivery	Same attempt, document and URL; request not aborted; attempt not expired. Explicit candidate choice is routed to that exact document ID.
Before writing	Page visible; same target elements and constraints; inputs usable and still empty. Code shape and field format agree.
After writing	Use the native input setter, emit input/change events and verify the resulting value. Stop polling and clear candidate references after fill acknowledgement.

When the attempt stops

You start typing

Preserve your text and cancel automatic filling.

The page changes

Navigation, replacement fields or a changed URL invalidate the old task.

Access or time runs out

Disconnect, disable, removed permission and the deadline stop further processing.

Submission boundary: extension code never calls form submit or clicks Verify. However, a website may automatically submit or transmit the OTP when it receives input/change events. Manual submission cannot be guaranteed for every site.

Code: src/background.ts; src/background/messages.ts; src/background/coordinator.ts; src/content/index.ts and autofill.ts.

The limits you should understand

A control reduces a particular risk. It cannot make every surrounding system trustworthy.

Risk	Implemented protection	Residual limit
Wrong or stale code	Parent-domain matching, short time window, compatible format, ambiguity handling.	No authenticated sender or server challenge binding. A malicious same-parent host can still be risky.
Malicious approved website	Per-host opt-in and final target checks.	Site scripts can read the filled OTP and can shape fields. Only approve hosts you trust.
Malicious email / input	Text-only extraction, bounded parsing, supported-pattern whitelist.	No dedicated adversarial audit. Unusual formats may be refused or misinterpreted.
Unsafe script loading	Manifest V3 with extension-page script-src self; bundled JavaScript, without remote script execution.	Content security policy reduces script injection risk. It does not certify the logic or protect against a malicious replacement build.
Stolen authorization	Background token use, Chrome-managed cache, no app token persistence/logging.	A live bearer token, device malware or a malicious update could expose broad mailbox data.
Sensitive data retained	No OTP history; attempt cleanup; trusted storage contexts.	Metadata remains temporarily. Cleanup is not secure memory wiping and does not remove data from Google or the site.
Revocation failure	Stop processing locally, request revocation, clear cached authorization.	A network failure can leave the Google grant active; remove it in Google Account connections.

Use it with clear boundaries

- Approve only the exact hosts needed for your task. Remove old site grants in Settings.
- Treat an unexpected code, request or domain mismatch as a reason to stop and verify manually.
- Protect your device, Chrome profile and developer account; review updates to unpacked files.
- Disconnect when finished. If revocation cannot be confirmed, use Google Account connections. Uninstalling alone should not be relied on to revoke Google access.

Code-based threat review; not a penetration test or quantified risk model. Platform guidance: [5], [6].

Evidence, with its limits

Passing tests support specific behaviors. They do not prove the extension is attack-proof.

83 unit / integration tests

Parser, matching, permissions, routing, Gmail behavior, orchestration and local identity sync.

17 Chromium browser tests

Controlled inputs and Gmail/Identity responses; includes the production setup-guidance scenario.

Evidence	What it establishes
Latest configured npm run check	Strict types, lint, formatting, 83 tests, production build and 17 browser tests passed on 21 October 2026.
Build / package inspection	13 installed and ZIP files matched the configured build byte-for-byte. Existing extension ID was preserved.
Cloud setup	Chrome Extension client, enabled Gmail API, owner test-user membership and saved gmail.readonly scope verified in Google Cloud.
Owner confirmation	Owner reported Gmail connected after reload and consent. Extension-page status was not independently inspected through automation.

Still required before broad release

- Controlled live OTP retrieval and filling on supported sites, including unusual forms, resend and simultaneous requests.
- Live consent cancellation, expired/revoked access, Google-side revocation and administrator restrictions.
- Independent security review, adversarial sender/DOM tests and representative accuracy/latency measurements.
- Applicable Google restricted-scope review, published privacy disclosures and Chrome Web Store review. Testing status is not public release approval.

Next acceptance check: on a trusted approved site, request a fictional/test-account verification email, confirm the expected field receives the right code, then check cancellation and disconnect. Do not use a real financial transaction as a test.

Evidence: docs/validation.md and executed local checks from the OAuth setup. Platform release requirements: [6].

Where these explanations come from

Reviewed against the current workspace source and official Google/Chrome documentation.

Implementation map

Topic	Source modules
Google calls and tokens	src/background/gmail.ts; src/background.ts
Discovery and extraction	src/content/detector.ts; src/core/email-parser.ts
Matching and timing	src/core/matcher.ts; src/core/lifecycle.ts; src/core/types.ts
Trusted delivery and storage	src/background/coordinator.ts; messages.ts; permissions.ts; settings.ts; src/content/index.ts; autofill.ts
Permissions and validation	manifest.config.mjs; src/core/validation.ts; docs/validation.md

Official platform references

[1] Gmail API scopes

https://developers.google.com/workspace/gmail/api/auth/scopes

[2] Chrome Identity API

https://developer.chrome.com/docs/extensions/reference/api/identity

[3] Chrome storage API

https://developer.chrome.com/docs/extensions/reference/api/storage

[4] Content scripts and isolated worlds

https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts

[5] Chrome extension security guidance

https://developer.chrome.com/docs/extensions/develop/security-privacy/stay-secure

[6] Google restricted-scope verification

https://developers.google.com/identity/protocols/oauth2/production-readiness/restricted-scope-verification

Scope of this review: v0.1.0 local implementation, extension ID pfhmngabnonjbbmomndimnkebibdmjkh. The report contains no real OTP, access token, private key or inbox content. Diagrams show design and policy; they are not telemetry, security ratings or measured production performance.

Platform references accessed 21 October 2026. Changes to source, permissions or data flows require a fresh review.